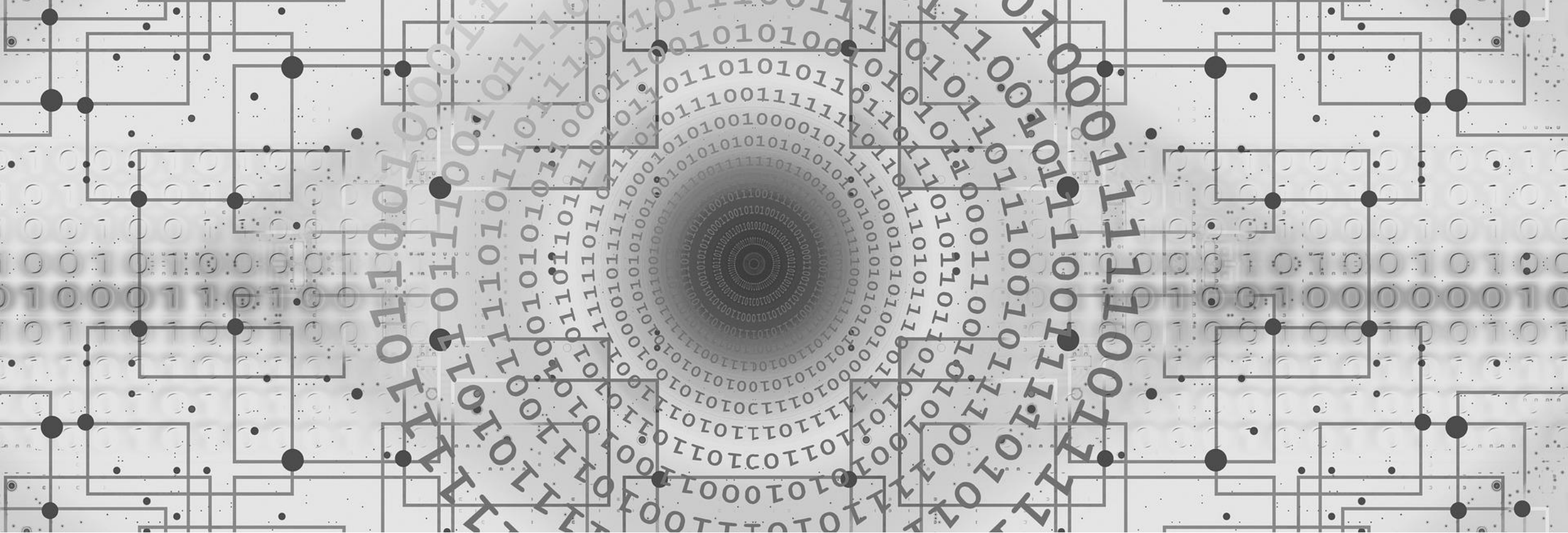




Homework-9 : Synchronous and Asynchronous FIFO Modeling

ECE-111 Advanced Digital Design Project

Outline

❑ First let's learn FIFO Concepts

- What is FIFO and how FIFO works
- Types of FIFO
 - Synchronous FIFO
 - Asynchronous FIFO
- Difference between synchronous and asynchronous FIFO
- Synchronous and Asynchronous FIFO full and empty flag generation
- Why Asynchronous FIFO is required for multi-bit signal crossing fast to slow clock domain
- FIFO Depth Calculation

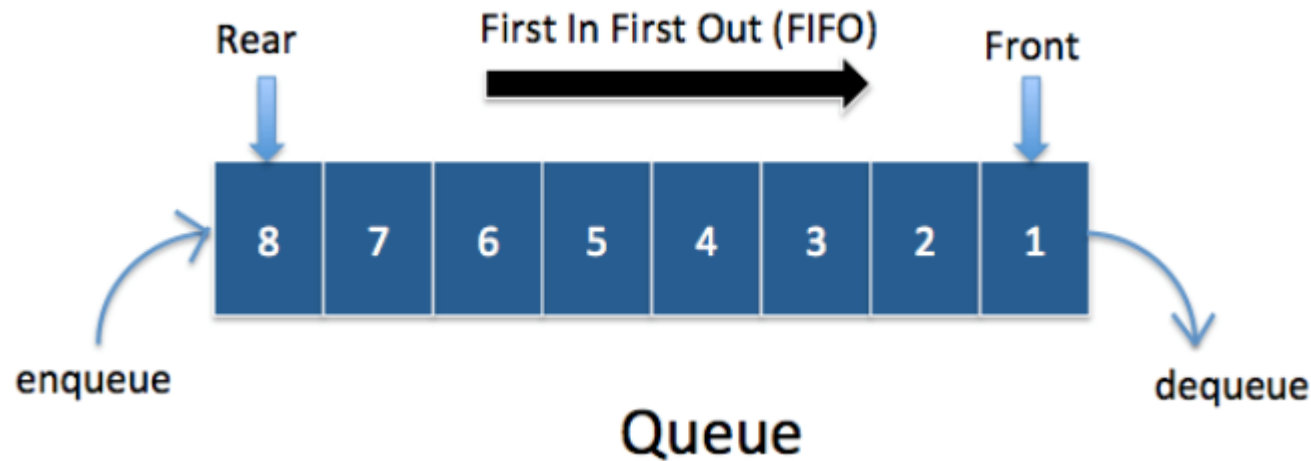
❑ Homework-9 :

- 9a: Requirements for Synchronous FIFO
- 9b: Requirement for Asynchronous FIFO

What is a FIFO ?

❑ FIFO means **First-In First-Out**

- A FIFO is a structure used in hardware or software application when one needs to buffer data
- FIFO is a queue, with first data in comes out first
- FIFO contains memory structure to store incoming data



FIFO Concepts

- ❑ FIFO is implemented using a **circular memory buffer** to store data element
- ❑ FIFO has a write pointer and read pointer in order to write data to the current memory address and read data from the memory address
- ❑ The maximum number of elements that can be stored inside FIFO memory is known as the **FIFO depth**
- ❑ Each element inside FIFO memory can be 1 or more number of bits. It is known as **DATA WIDTH**
- ❑ In Figure-A, since write pointer (wr_ptr) and read pointer (rd_ptr) is pointing to same address location. This means FIFO is empty
- ❑ In Figure-B, write pointer has advanced and it is pointing to memory address location '3'. It means three data elements E1, E2, E3 were pushed into the FIFO. Since read pointer is pointing to 0th address location, it means no data has been popped (read) out of FIFO yet.

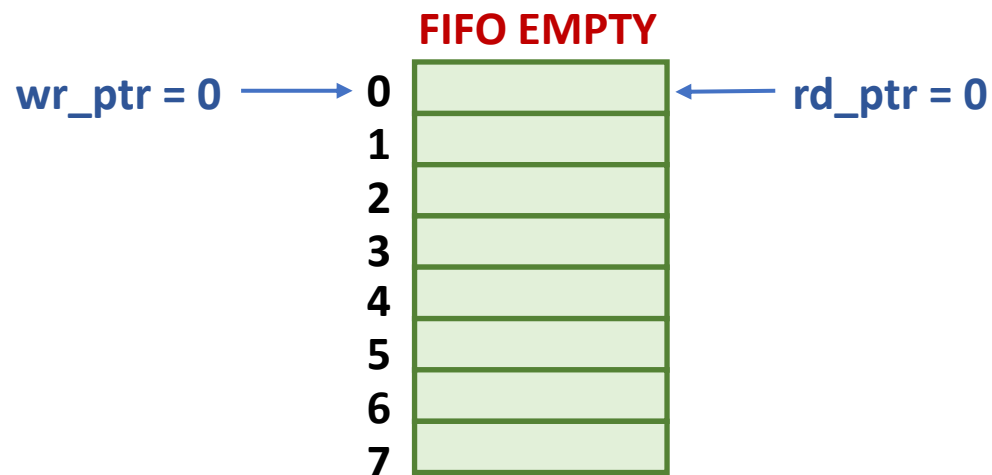


Figure-A

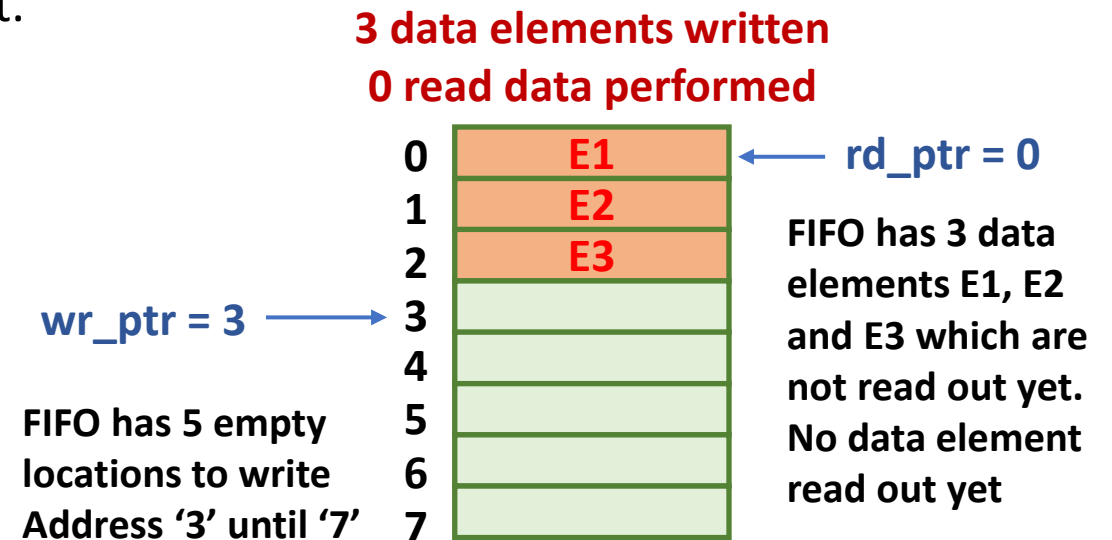
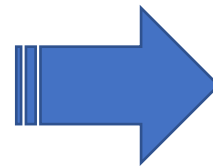
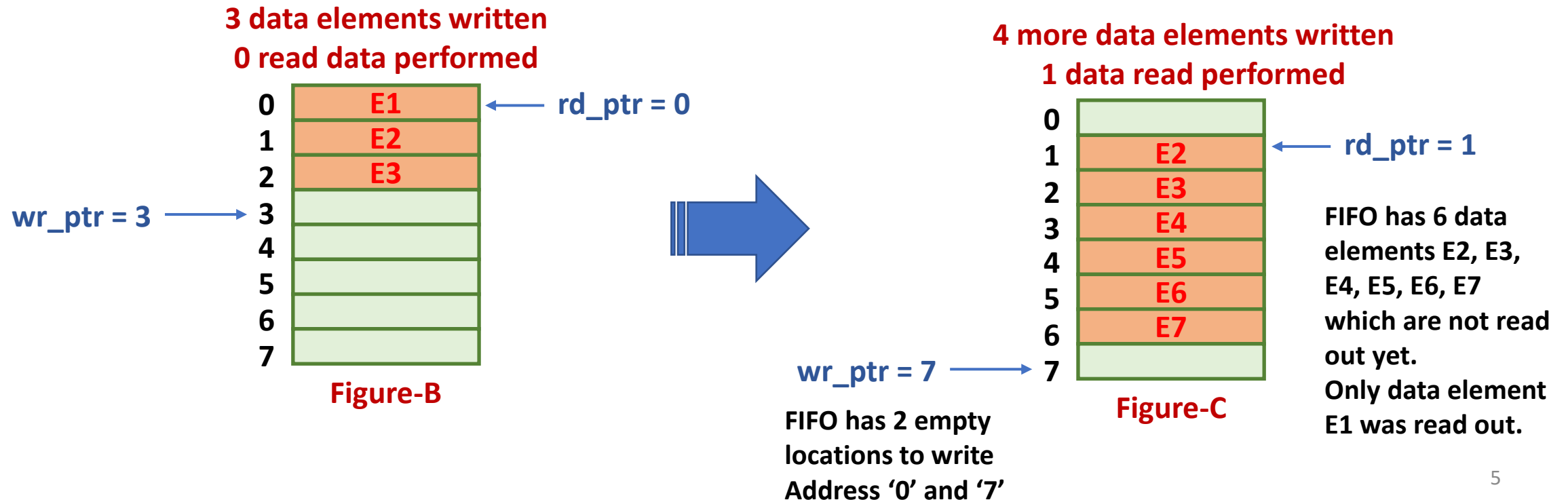


Figure-B

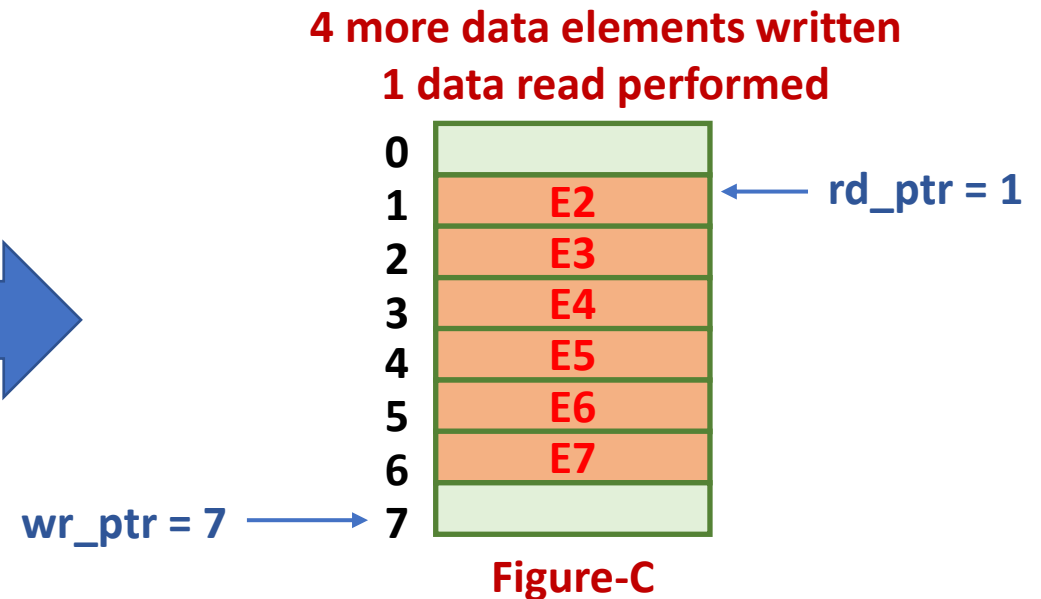
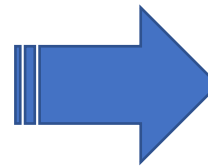
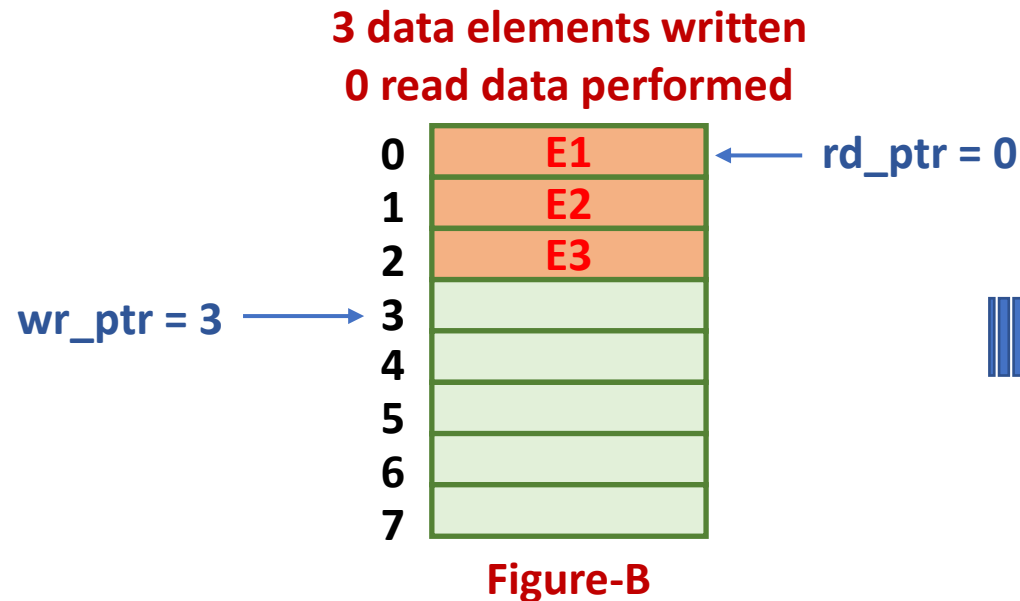
FIFO Concepts

- ❑ In Figure-C, write pointer has advanced and it is pointing to memory address location '7'
 - It means 4 new data elements E4, E5, E6, E7 were pushed into the FIFO.
- ❑ In Figure-C, read pointer has advanced to address location '1'
 - it means data from address location '0' which was E1, was read out from the FIFO memory
 - Remember, E1 was the first data entered into the FIFO and now the first data to exit



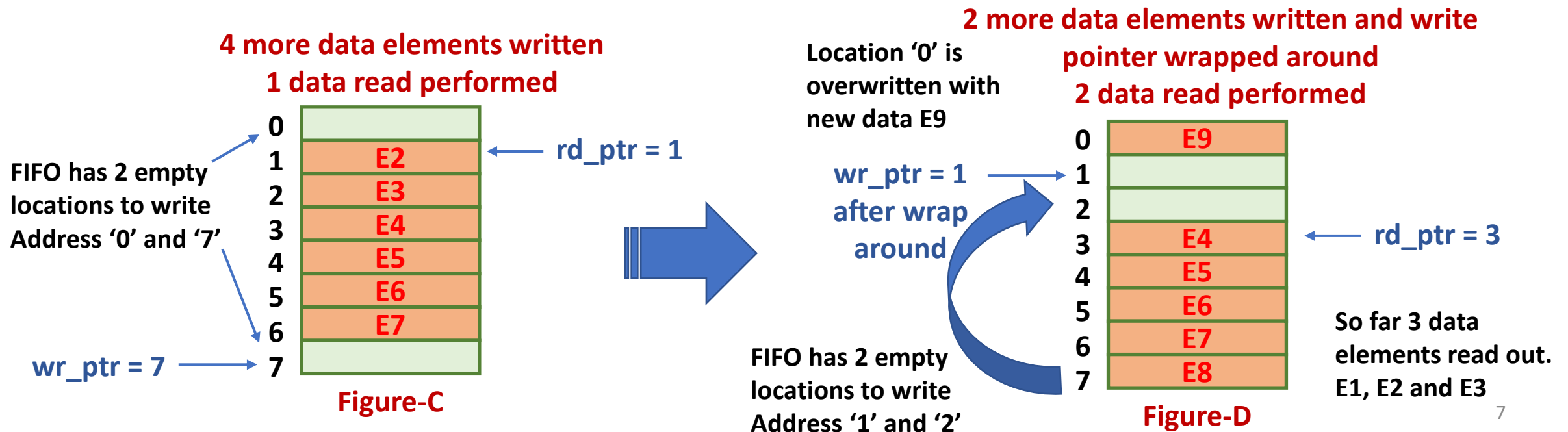
FIFO Concepts

- ❑ In Figure-C, write pointer has advanced and it is pointing to memory address location '7'
 - It means 4 new data elements E4, E5, E6, E7 were pushed into the FIFO.
- ❑ In Figure-C, read pointer has advanced to address location '1'
 - it means data from address location '0' which was E1, was read out from the FIFO memory
 - Remember, E1 was the first data entered into the FIFO and now the first data to exit



FIFO Concepts

- ❑ In Figure-D, write pointer has advanced and it has wrapped around to point to location '1'
 - It means 2 new data elements E8, E9 were pushed into the FIFO.
 - FIFO write pointer was able to wrap around since there were 2 new elements were to be pushed into FIFO. Location '7' was available and location '0' was freed up since E1 data from location '0' was read out previously.
 - Since write and read pointers can wrap around, FIFO memory is known as **circular buffer**
- ❑ In Figure-D, read pointer has advanced to address location 'd'
 - it means data from address location '1' and '2' which were E2 and E3 were read out



FIFO Concepts

❑ FIFO Full and FIFO Empty can look the same !

- In Figure-A and Figure-E, both **wr_ptr** and **rd_ptr** are pointing to same location '0'
- Pointing to same address location does not always mean that FIFO is empty

❑ To determine if FIFO is empty or FIFO is full, additional information is required to differentiate

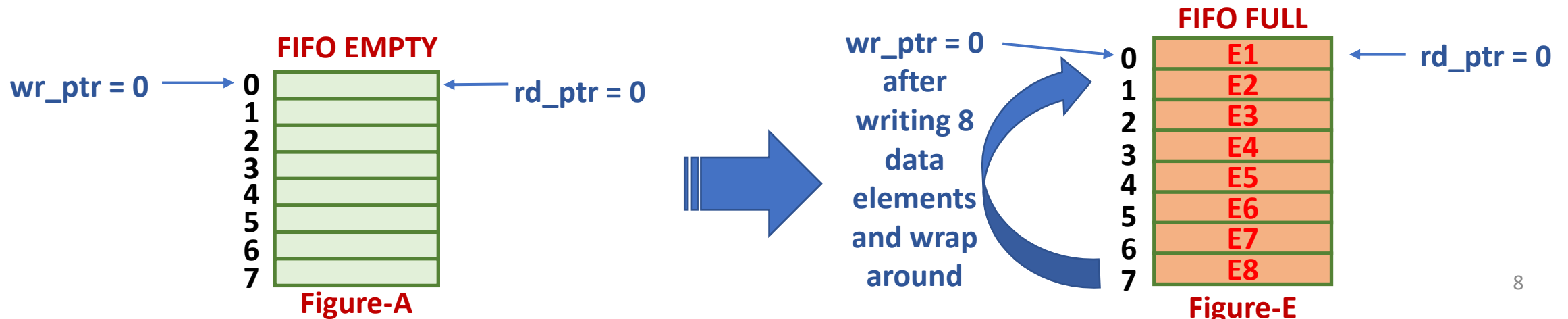
- Typically additional address bit is added to the MSB of write and read pointers
- If FIFO_DEPTH = 8, then wr_ptr and rd_ptr width is 3 bit each
- In such case, 4th bit is added to wr_ptr and rd_ptr. This is called as a “**wrap**” around bit

❑ FIFO is Empty if ($wr_ptr[3:0] == rd_ptr[3:0]$)

- Wrap around MSB bit of write and read pointer and remaining bits have same value

❑ FIFO is Full if ($(wr_ptr[3] \neq rd_ptr[3]) \ \&\& \ (wr_ptr[2:0] == rd_ptr[2:0])$)

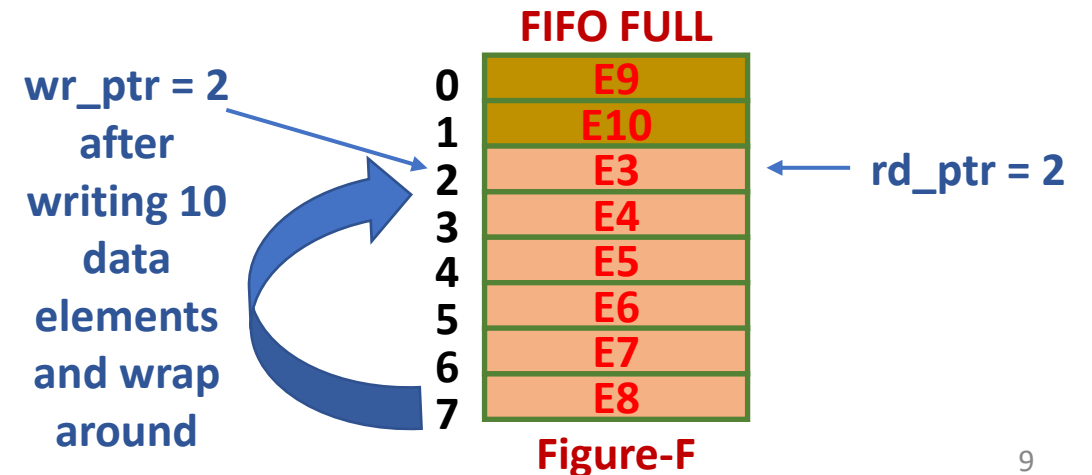
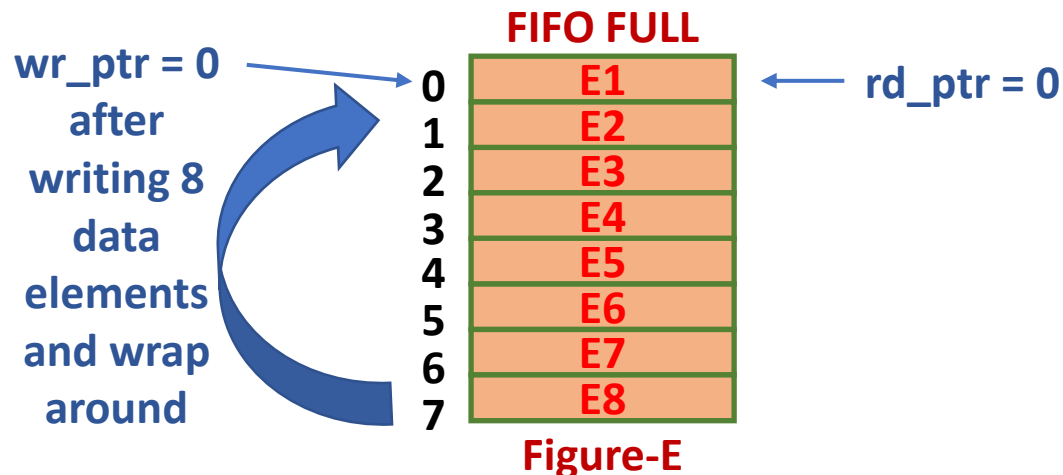
- MSB wrap around bit's or write and read pointer are different and remaining bits are same



FIFO Concepts

❑ Both Figure-E and Figure-F represents FIFO Full Condition

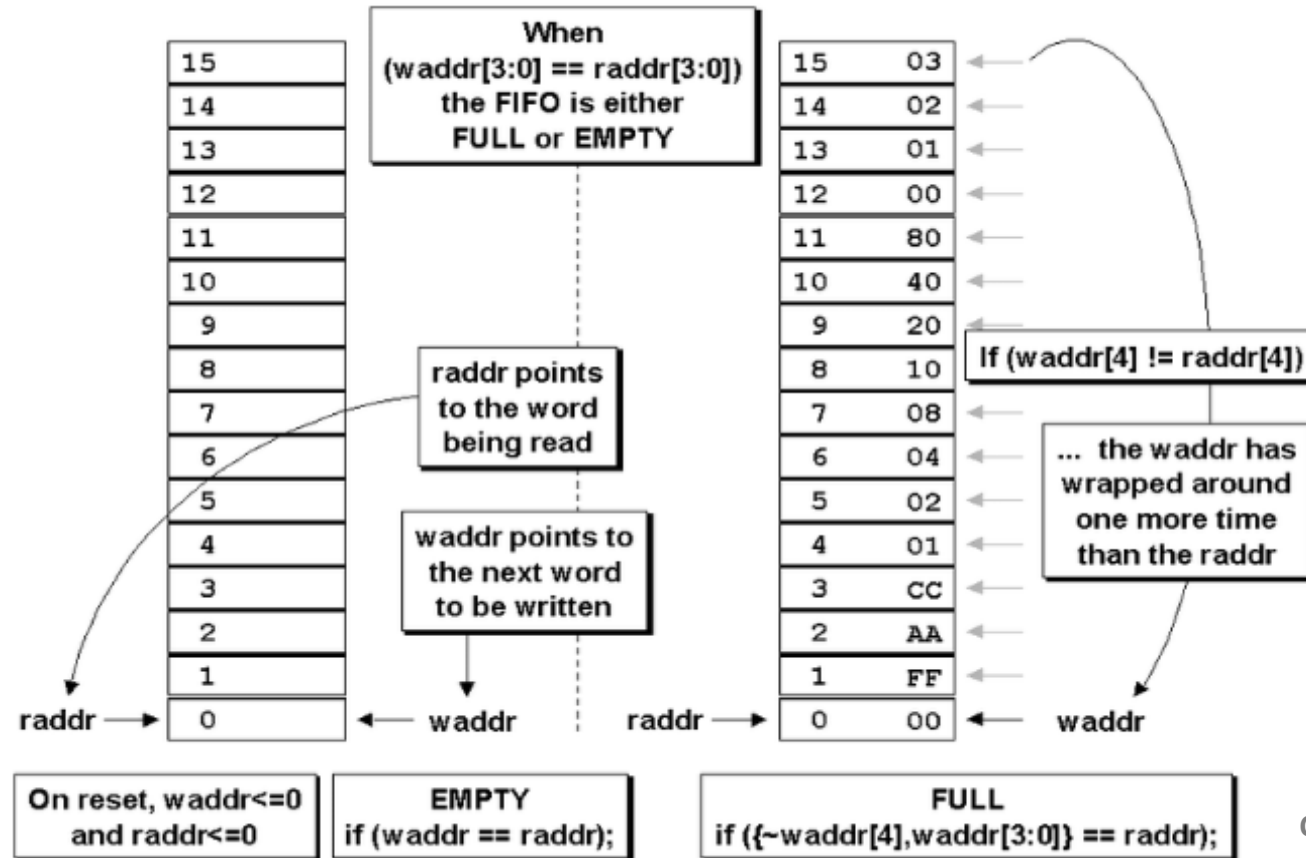
- In Figure-A and Figure-E, both **wr_ptr** and **rd_ptr** are pointing to same address locations
- In Figure-A, 8 data elements were written and 0 were read out. Hence FIFO is full as max occupancy in FIFO memory is 8
- In Figure-F, 10 data elements were written and 2 were read out. Hence FIFO is full as max occupancy in FIFO memory is 8
- In Figure-E, **wr_ptr** = 4'b1000 and **rd_ptr** = 4'b0000
 - **FIFO_FULL = 1**, since (**wr_ptr**[4] != **rd_ptr**[4] && **wr_ptr**[3:0] == **rd_ptr**[3:0])
- In Figure-F, **wr_ptr** = 4'b1010 and **rd_ptr** = 4'b0010
 - **FIFO_FULL = 1**, since (**wr_ptr**[4] != **rd_ptr**[4] && **wr_ptr**[3:0] == **rd_ptr**[3:0])



FIFO Concepts

❑ FIFO Full and FIFO Empty differentiation using MSB bit (wrap around Bit)

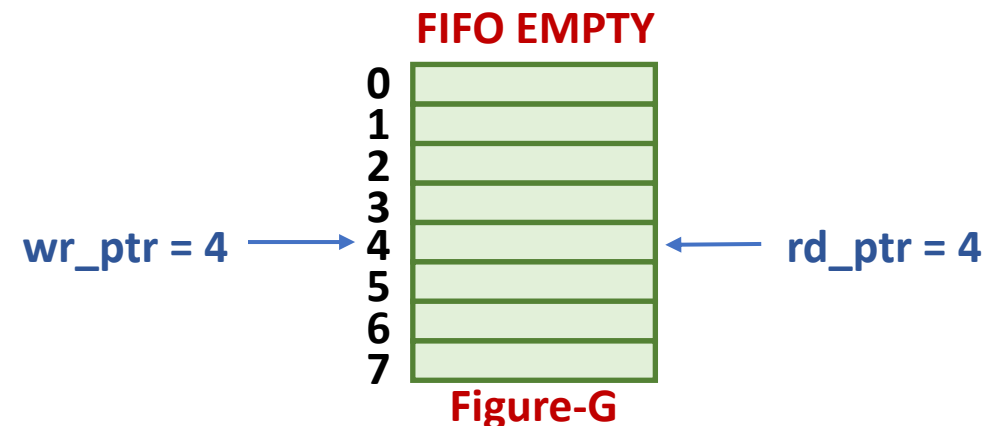
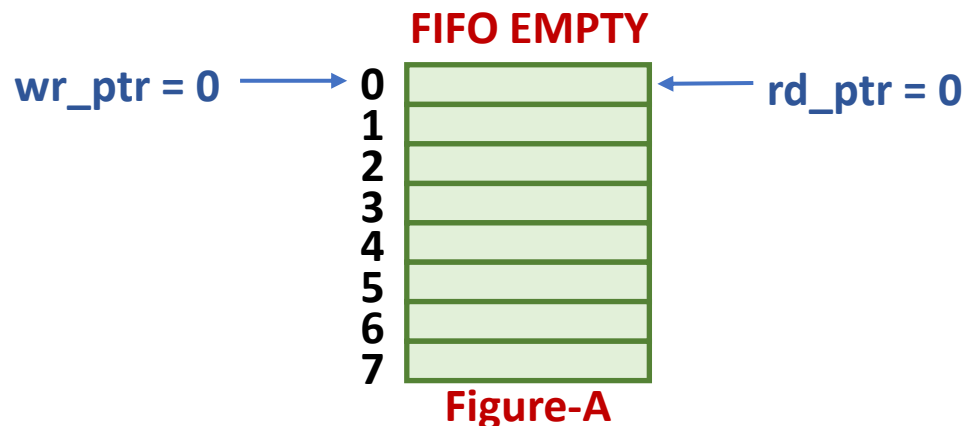
- Note : waddr and raddr in diagram is same as write and read pointers (wr_ptr and rd_ptr)
- Below mentioned example is with FIFO_DEPTH -16 with raddr and waddr have 5 bits each
 - MSB bit position waddr[4] and raddr[4] is the wrap around bit to differentiate between full and empty condition. Hence FIFO depth is still $2^4 = 16$ and not $2^5 = 32$.
 - In another words, only 16 data elements can be stored in FIFO Memory since MSB bit 4 is a wrap around bit



FIFO Concepts

❑ Both Figure-A and Figure-G represents FIFO Empty Condition

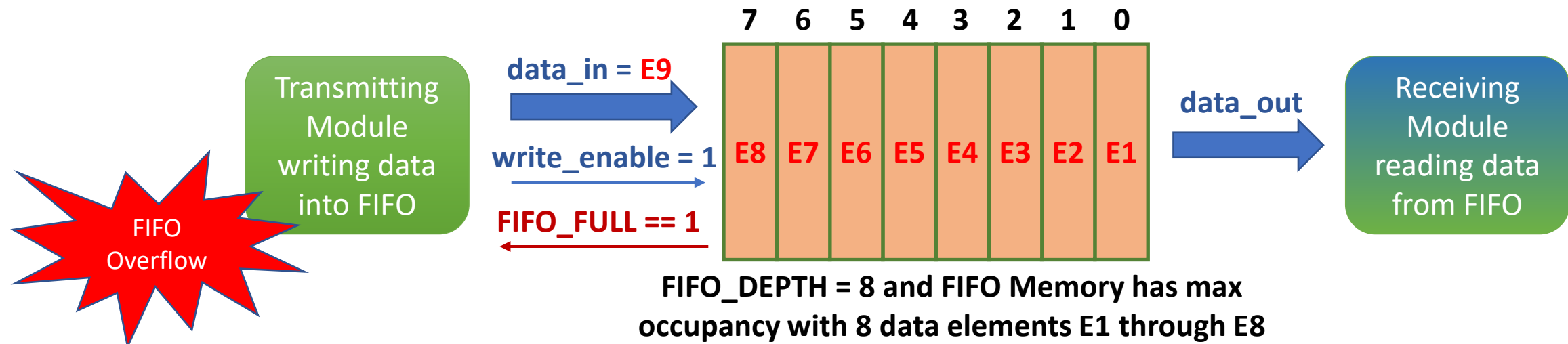
- In Figure-A and Figure-G, both **wr_ptr** and **rd_ptr** are pointing to same address locations
- In Figure-A, 0 data elements were written and 0 were read out. Hence FIFO is empty since all 8 locations in FIFO memory are available to be written
- In Figure-G, 4 data elements were written and 4 were read out. Hence FIFO is empty since all 8 locations in FIFO memory are available to be written
- In Figure-A, **wr_ptr = 4'b0000** and **rd_ptr = 4'b0000**
 - **FIFO_EMPTY = 1**, since (**wr_ptr[4:0] == rd_ptr[4:0]**)
- In Figure-G **wr_ptr = 4'b0100** and **rd_ptr = 4'b0100**
 - **FIFO_EMPTY = 1**, since (**wr_ptr[4:0] == rd_ptr[4:0]**)



FIFO Concepts

❑ FIFO Overflow :

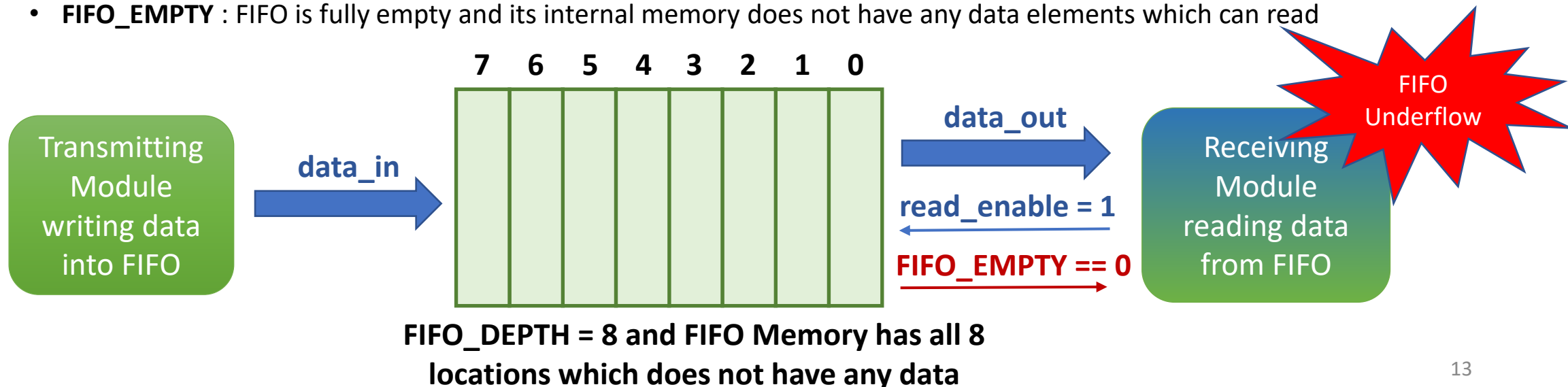
- When FIFO internal memory is full and any attempt to write a new data element to FIFO memory is called FIFO overflow condition
- Overflow expression is : $(\text{FIFO_FULL} == 1) \ \&\& \ (\text{write_enable} == 1)$. **See below mentioned Figure.**
- It is the responsibility of transmitting module to ensure it does not write to FIFO when it is full
- Typically FIFO designs provides FIFO_FULL signal which gives indication to transmitting module if FIFO has any room to write or not
- Additionally FIFO designs provides such more flags to indicate memory occupancy information
 - **FIFO_ALMOST_FULL** : FIFO internal memory has 1 location remaining to write before it is full. Acts like an early full indication
 - **FIFO_HALF_FULL** : FIFO internal memory has 50% occupancy
 - **FIFO_FULL** : FIFO is full and its internal memory has max occupancy



FIFO Concepts

❑ FIFO Underflow :

- When FIFO internal memory is fully empty and any attempt to read any location of FIFO memory is called FIFO underflow condition
- Underflow expression is : $(\text{FIFO_EMPTY} == 0) \ \&\& \ (\text{read_enable} == 1)$. **See below mentioned Figure.**
- It is the responsibility of receiving module to ensure it does not read FIFO memory when it is empty
- Typically FIFO designs provides FIFO_EMPTY signal which gives indication to receiving module if FIFO has any data element to read or not.
- Additionally FIFO designs provides such more flags to indicate memory occupancy information
 - **FIFO_ALMOST_EMPTY** : FIFO internal memory has 1 data element remaining to be read before it is fully empty. Acts like an early empty indication
 - **FIFO_HALF_EMPTY** : FIFO internal memory has 50% occupancy
 - **FIFO_EMPTY** : FIFO is fully empty and its internal memory does not have any data elements which can read



Types of FIFO

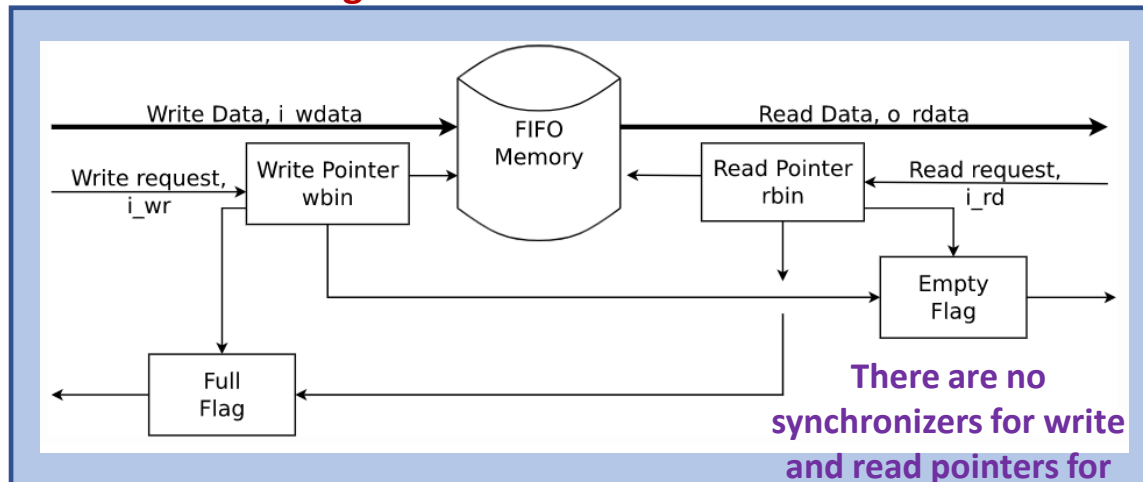
❑ There are 4 kinds of FIFO

- Shift registers FIFO
- Exclusive Read/Write FIFO's
- Concurrent Read/Write FIFO's
 - **Synchronous and Asynchronous FIFO**

❑ Basic difference between Synchronous and Asynchronous FIFO :

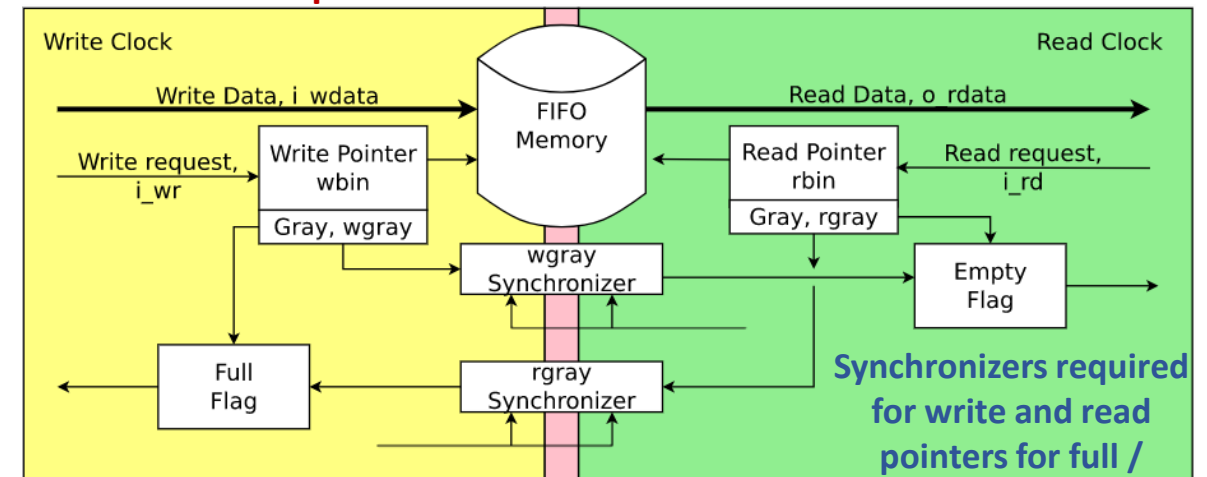
- In case of synchronous FIFO both write and read operation is performed on the same clock
- In case of asynchronous FIFO write operation and read operation of asynchronous FIFO are asynchronous to each other.
 - **Write and read clock can run independently with same or different frequency !**

Single Clock for write and Read



Synchronous FIFO full / empty generation

Separate Clock for write and read

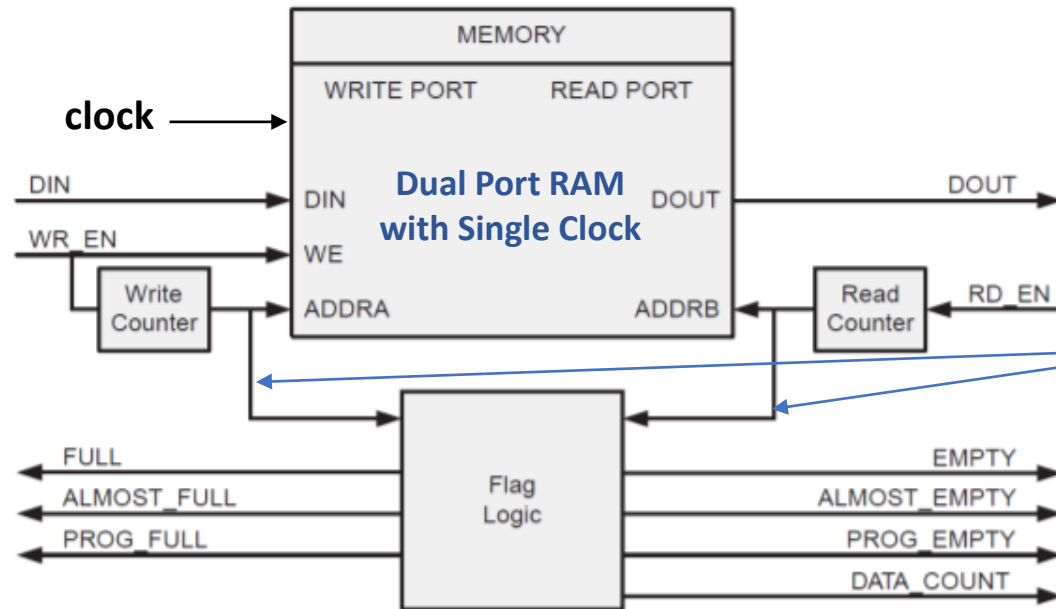


Asynchronous FIFO

Synchronous FIFO

Single clock used
for both read and
write operation !

Dual Port RAM
supports
simultaneous write
and read operation !



There are no
synchronizers for
write and read
address pointers for
full / empty Flag
generation !

FIFO Operation :

- Within FIFO there is a has dual port memory block for storage. **It allows simultaneous write and read operation !**
- FIFO has an input data in port (DIN) and an output read port (DOUT).
- Each data port has its own associated pointers which points to a location in the memory
- After a FIFO reset both the write and read pointers will be at the first memory location within the FIFO.
- Each write operation will cause the write pointer to increment to the next location in memory
- Each read operation will cause the read pointer to increment to the next location
- FIFO has full and empty signals indicating whether it has no empty location for any new data to be stored (full condition) or it has no data available for reading (empty condition)
- Full marker prevents overriding of existing data. This is known as fifo overrun or overflow condition
- Empty marker prevents reading junk data, when it is empty. This is known as fifo underrun or underflow condition

Homework-9a : Synchronous FIFO

❑ Review and Study SystemVerilog RTL model for M-bit width and N-depth Synchronous FIFO :

- Review System Verilog code for Synchronous FIFO provided in Lab folder
- Review SystemVerilog code for dual port ram in Lab folder
- Synthesize sync_fifo model and run simulation using testbench provided
- Review synthesis results (resource usage and RTL netlist/schematic)
- Review input and output signals in simulation waveform.
 - Review dual_port_ram input and output signals in waveform to understand how write and read operations are performed
 - Review shift_register instances input and output signals in waveforms to understand how flipflop synchronizer works
 - Review sync_fifo input and output signals in waveform and understand write and read operations are performed

❑ Homework Report should include :

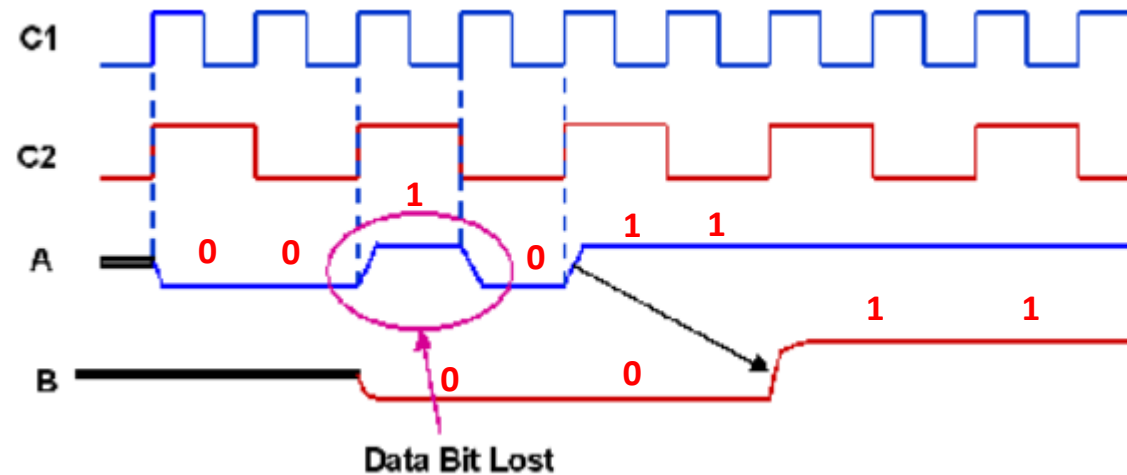
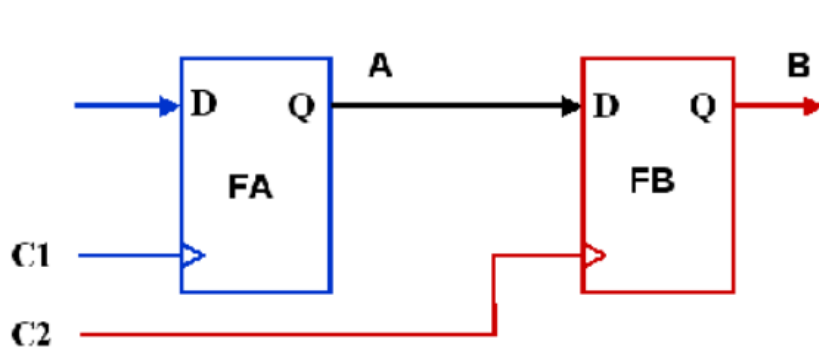
- SystemVerilog design code snapshot for sync_fifo, shift_register and dual_port_ram modules
- Synthesis resource usage snapshot generated from Quartus for async_fifo top level module
- Simulation snapshot and explain simulation result of sync_fifo :
 - Describe how data is sent to sync fifo and read from sync_fifo, Explain full and empty flags are generated, explain how sync fifo works
 - Explain role of dual_port_ram in sync_fifo design and how read and write operation is performed to dual_port_ram
- Explanation of FPGA resource usage in report is not required.

❑ Lab9 folder includes :

- sync_fifo folder which includes
 - Design template code for sync_fifo, dual_port_ram and shift_register modules
 - Full testbench code for sync_fifo
- For learning purpose, student can change the stimulus in initial block in testbench files

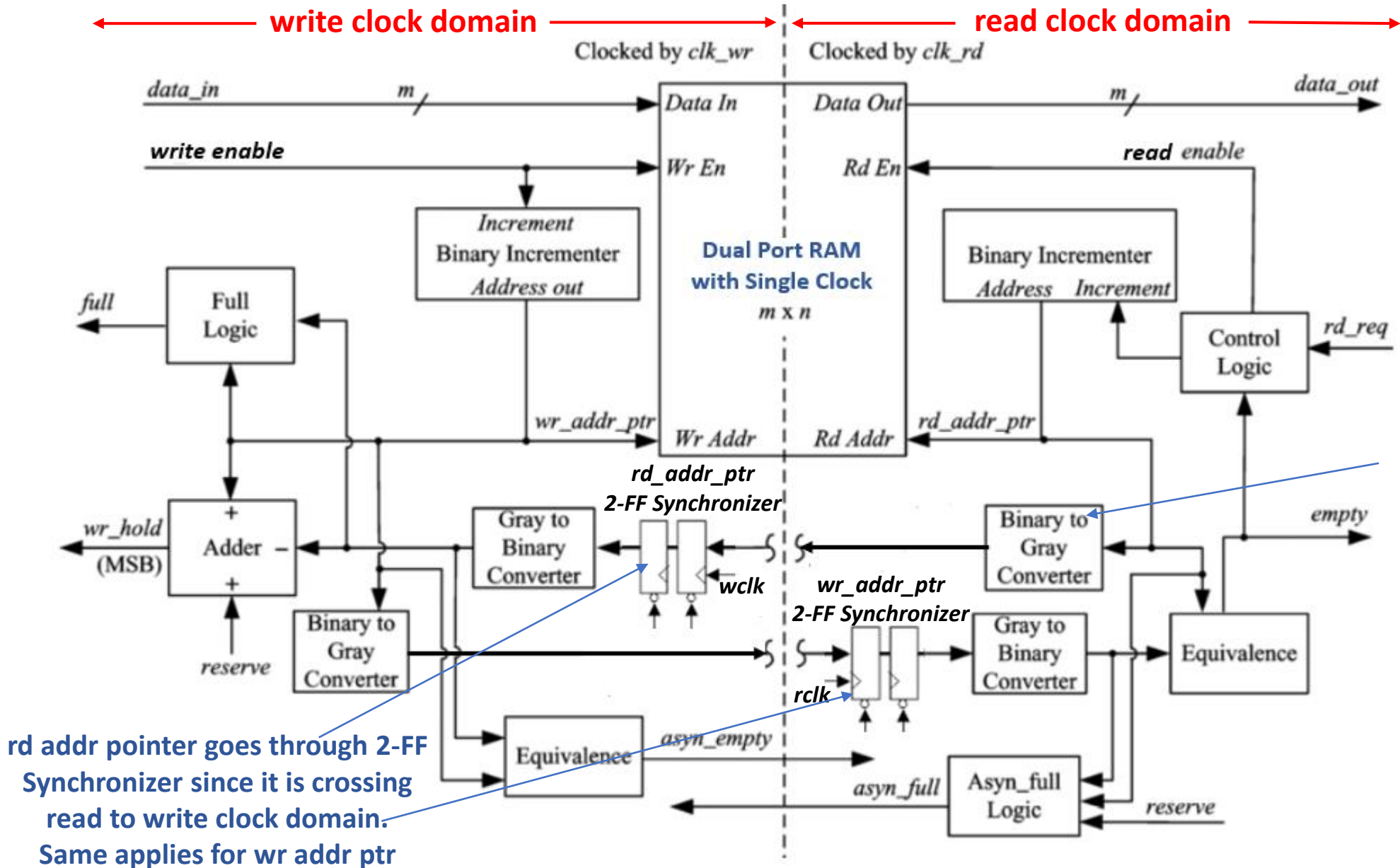
Why Async FIFO for Signal Crossing Fast Clock Domain to Slow Clock Domain

- ❑ Flipflop-A clock frequency > Flipflop-B clock frequency : 2FF synchronizer is not sufficient to avoid data loss !



- ❑ C1 is two times faster than C2 and there is no phase difference between C1 and C2
- ❑ If signal A is changing rapidly, say for example signal A sequence is "**001011**", output from FB in C2 clock domain will be "**0011**".
 - Here the third data value in the input sequence which is "**1**" is lost → **data loss**
 - For some circuits data loss is not acceptable
- ❑ **Two or three Flipflop synchronizer technique will avoid data loss, instead either of the techniques can be used to avoid data loss when data is sent from fast to slow clock domain :**
 - **Storage between FA and FB using Asynchronous FIFO**
 - Handshake Synchronization technique between FA and FB

Asynchronous FIFO



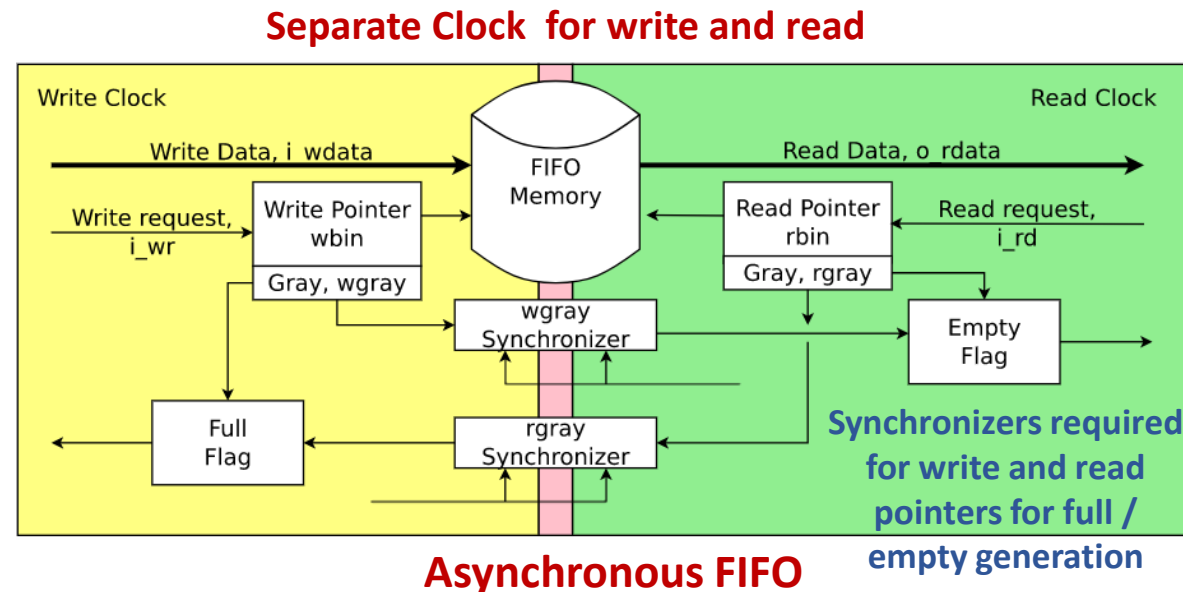
Dual Port RAM with Single clock is used
Since write is synchronous and read is asynchronous !

Before sending read pointer and write pointer to other clock domain, it is converted to gray coded value. In Gray value only 1 bit changes during counting. This counter update strategy minimizes the error when counter value passes from one clock domain to another

Asynchronous FIFO

❑ Asynchronous FIFO write and read operation is similar to synchronous FIFO except :

- In case of asynchronous FIFO there are 2-FF synchronizers are used to synchronize write pointer coming from write clock domain to read clock domain. This is required for fifo empty flag generation.
- Similarly, 2-FF synchronizers are used to synchronize read pointer coming from read clock domain to write clock domain. This is required for fifo full flag generation.
- Additionally before write and read pointers are fed to the synchronizer, both write and read pointer binary count values are converted to gray encoded values.
 - In gray encoded value only 1 bit changes during counting. This counter update strategy minimizes the error when counter value passes from one clock domain to another

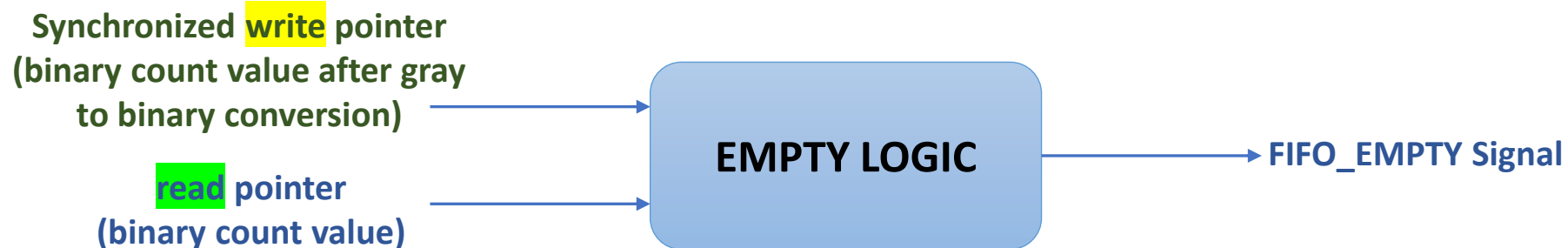


Asynchronous FIFO Full and Empty Generation



```
assign fifo_full = ((wr_ptr[ADDR_WIDTH] != rd_ptr_binary2[ADDR_WIDTH])  
    && (wr_ptr[ADDR_WIDTH-1:0] == rd_ptr_binary2[ADDR_WIDTH-1:0])) ? 1 : 0;
```

Note : rd_ptr_binary2 is synchronized read pointer binary value



```
assign fifo_empty = (wr_ptr_binary2 == rd_ptr) ? 1 : 0;
```

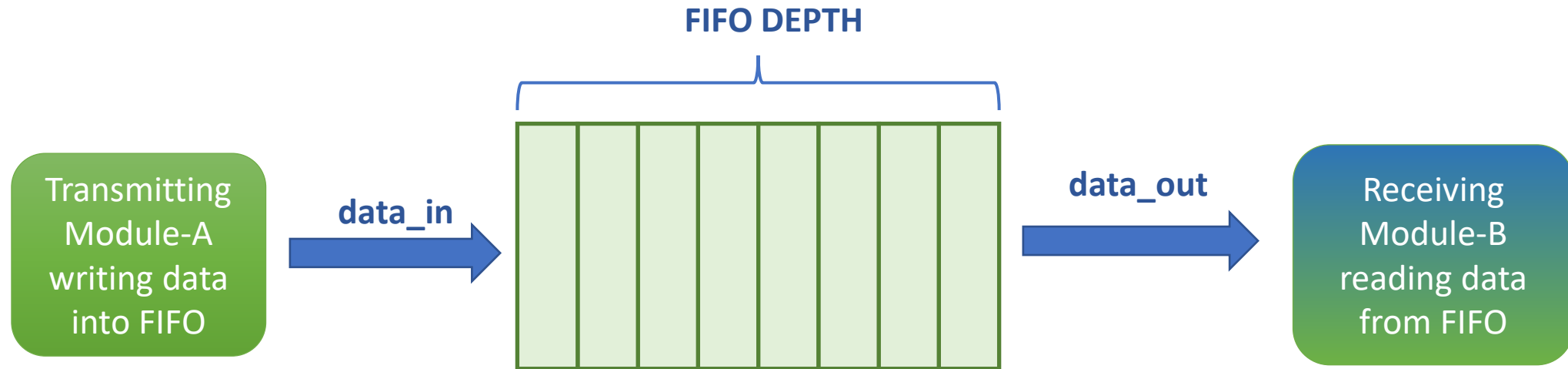
Note : wr_ptr_binary2 is synchronized write pointer binary value

Asynchronous FIFO Depth Calculation

- ❑ To compute depth of asynchronous FIFO, following parameters are required to be known :
- Write and read clock frequency
 - Amount of data to be written (also know as Burst Length)
 - Rate at which data is written to the memory and rate at which data is read from the memory.

Example :

- There is 1 cycle of idle between two consecutive writes from module A to the FIFO
- There is 3 cycle of idle between two consecutive read request from module B to the FIFO



Asynchronous FIFO Depth Calculation

□ Case-1:

- $f_A > f_B$ with no idle cycles in both write and read.
- Writing frequency = $f_A = 80\text{MHz}$.
- Reading Frequency = $f_B = 50\text{MHz}$.
- Burst Length = No. of data items to be transferred = 120.
- There are no idle cycles in both reading and writing which means that, all the items in the burst will be written and read in consecutive clock cycles.

□ Solution:

- Time required to write one data item $(1 / 80 \text{ Mhz}) = 12.5 \text{ nSec}$
- Time required to write all the data in the burst = $120 * 12.5 \text{ nSec.} = 1500 \text{ nSec.}$
- Time required to read one data item $(1 / 50 \text{ Mhz}) = 20 \text{ nSec.}$
- For every 20 nSec, the module B is going to read one data in the burst.
- In a period of 1500 nSec, 120 no. of data items can be written.
- And the no. of data items can be read in a duration of 1500 nSec = $(1500 \text{ nSec} / 20 \text{ nSec}) = 75$
- The remaining no. of bytes to be stored in the FIFO = $120 - 75 = 45$.
- Hence the FIFO which has to be in this scenario must be capable of storing **45** data items.
- **Answer :** FIFO DEPTH = **45**

Asynchronous FIFO Depth Calculation

❑ Case-2:

- $f_A > f_B$ with no idle cycles in both write and read.
- Writing frequency = $f_A = 80\text{MHz}$.
- Reading Frequency = $f_B = 50\text{MHz}$.
- Burst Length = No. of data items to be transferred = 120.
- No. of idle cycles between two successive writes is = 1
- No. of idle cycles between two successive read is = 3

❑ Solution:

- The no. of idle cycles between two successive writes is **1** clock cycle. It means that, after writing one data, module A is waiting for one clock cycle, to initiate the next write. So, it can be understood that for every **2** clock cycles, one data is written.
- The no. of idle cycles between two successive reads is **3** clock cycles. It means that, after reading one data, module B is waiting for 3 clock cycles, to initiate the next read. So, it can be understood that for every **4** clock cycles, one data is read
- Time required to write one data item **2** * (1 / 80 Mhz) = 25 nSec
- Time required to write all the data in the burst = 120 * 25 nSec. = 3000 nSec.
- Time required to read one data item **4** * (1 / 50 Mhz) = 80 nSec.
- For every 80 nSec, the module B is going to read one data in the burst.
- In a period of 3000 nSec, 120 no. of data items can be written.
- And the no. of data items can be read in a duration of 3000 nSec = (3000 nSec / 80 nSec) = 37.5 (~37)
- The remaining no. of bytes to be stored in the FIFO = 120 – 37 = **83**. **Answer :** FIFO DEPTH = **83**

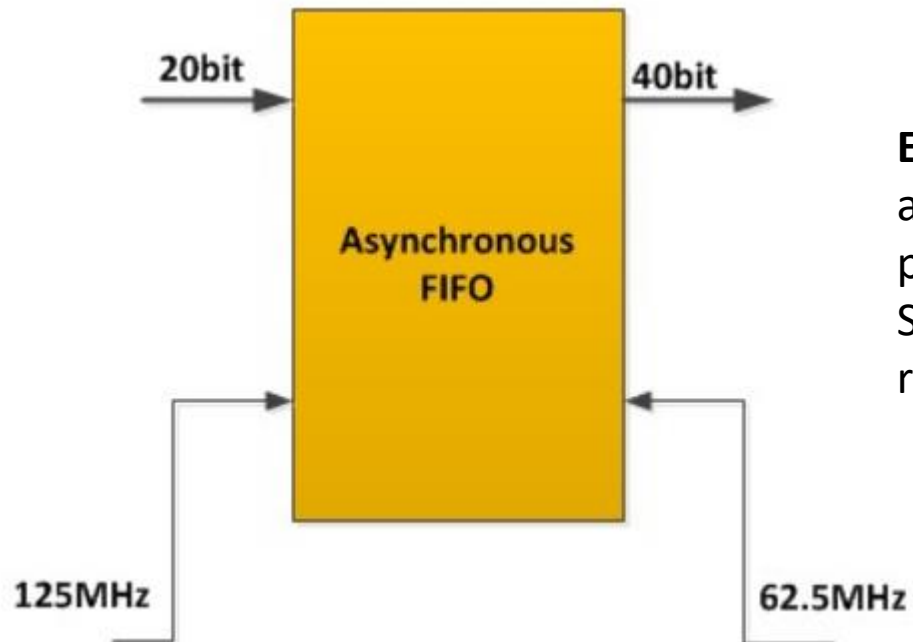
Asynchronous FIFO Depth Calculation

❑ In homework Lab, FIFO DEPTH can be fixed to 8 in testbench based on below mentioned assumption :

- Write clock time period is 20ns
- Read clock time period is 40ns
- Number of packets to write is 16
- No. of idle cycles between two successive writes is = 0
- No. of idle cycles between two successive reads is = 0
- $\text{FIFO Depth} = 16 - ((16 * 20\text{ns}) / 40\text{ns}) = 8$

Application of Asynchronous FIFO

- ❑ **Multi-bit data buses are synchronized across clock domain using Asynchronous FIFOs, provided :**
 - Incoming data rate is same as outgoing data rate.
 - That means **$(F1 \times DW1) = (F2 \times DW2)$** .
 - Where F1- Frequency of launching flop,
 - F2- Frequency of capturing flop,
 - DW1- write data width to FIFO, DW2- Read data width from FIFO.

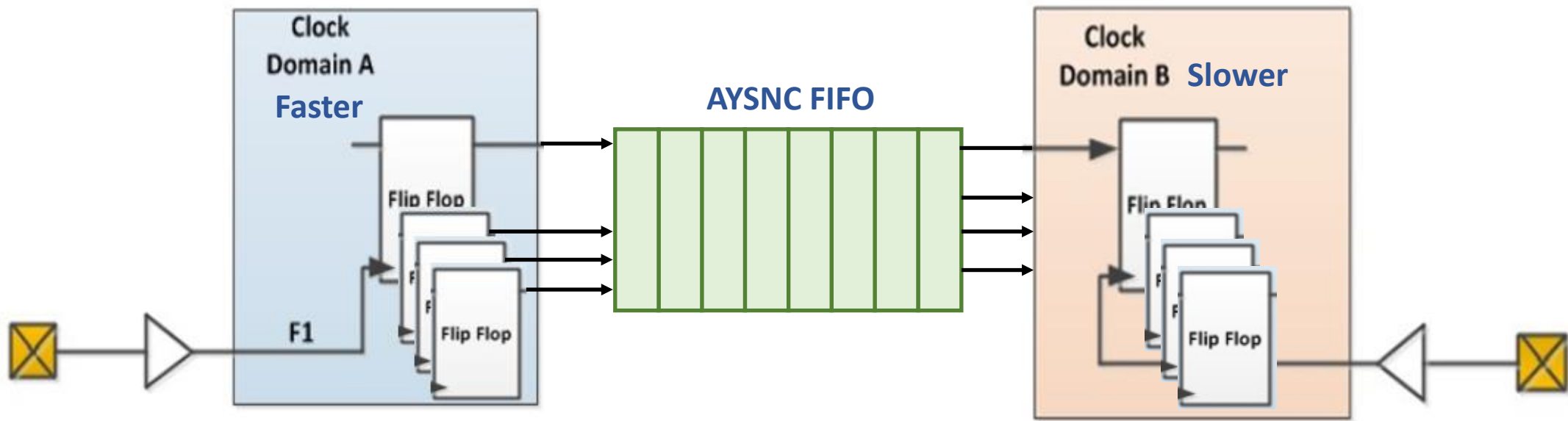


Example : 2.5Gigbps data path for PCIe Gen2. This data rate is achieved by using 20bit data path at 125MHz or 40bit data path at 62.5 MHz.

So, an asynchronous FIFO shown in the diagram will operate reliably without any data loss or duplication

Application of Asynchronous FIFO

- ❑ **Asynchronous FIFO are used for multi-bit bus transmission from faster to slower clock domain**
 - Using 2-FF or 3-FF synchronizer for signals crossing fast to slow clock domain can lead into data loss.
 - Instead asynchronous FIFO is used in such scenarios to prevent data loss
 - Designs where data loss is not acceptable when crossing faster to slow clock domain then asynchronous fifo can be used as a synchronizer



Homework-9b : Asynchronous FIFO

❑ Develop SystemVerilog RTL model for M-bit width and N-depth Asynchronous FIFO :

- Asynchronous FIFO RTL model should be configurable to set following mentioned parameters :
 - FIFO_DEPTH : This is number of data locations FIFO's internal memory can store. FIFO DEPTH value should be power of 2 (such as 2, 4, 8, 16, 32, and so on). Default value of FIFO DEPTH is set to 8 in testbench.
 - DATA_WIDTH : Width of each data element which can be stored in FIFO's internal memory. By default value is set to 32 data width in Testbench.
- Support different clock frequency for write and read. Write clock should be faster than read clock
 - In testbench write clock is set to 20 ns (which is 50 Mhz clock) and read clock is set to 40 ns (which is 25 Mhz)
 - In tesbench for above mentioned default write and read clock frequency set NUM_OF_PACKETS=8
 - NUMBER_OF_PACKETS : Number of data elements which needs to be transmitted through FIFO. By default set to '8' in testbench. Use default values provided in testbench which is derived for wr_clk=20ns and rd_clk=40ns
- Memory inside FIFO should be simple dual port memory with single clock.
 - Memory should support synchronous write and asynchronous read operation
 - Use dual_port_ram module provided in LAB folder which implements above mentioned requirement
- Use M-bit wide and N-Stage deep shift register for write and read pointer synchronization
 - Use shift_register module provided in LAB folder which can take M-bit of data and generate N-cycles delayed version of input data
- Synthesize async fifo RTL model and run simulation using testbench provided
- Review synthesis results (resource usage and RTL netlist/schematic)
- Review input and output signals in simulation waveform.
- Describe simulation behavior, FIFO operation and provide simulation snapshot, RTL code, resource estimation in final report

Homework-9b : Asynchronous FIFO

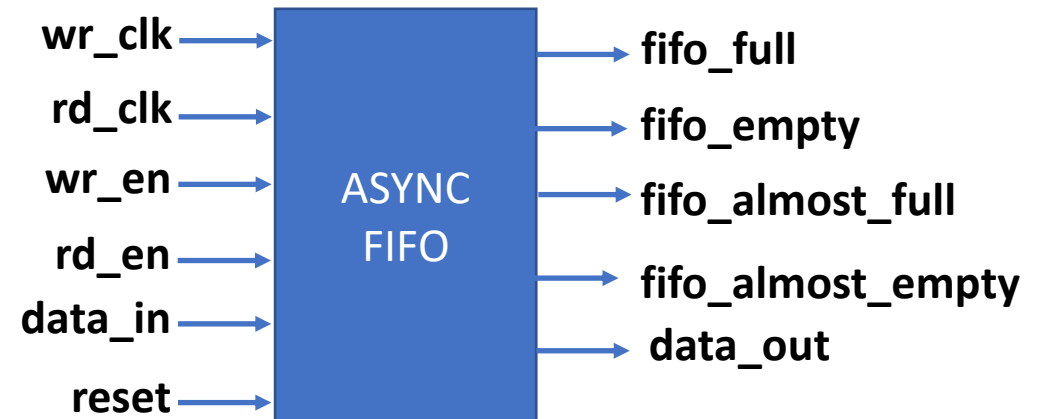
❑ Primary Ports for Asynchronous FIFO design :

- **input** logic **wr_clk, rd_clk** : Write and Read Clocks
- **input** logic **reset** : Asynchronous and active high reset
- **input** logic **wr_en** : write enable, if wr_en == 1, data gets written to FIFO Memory
- **input** logic **rd_en** : read_enable, if rd_en == 1, data gets read out from FIFO Memory
- **input** logic [DATA_WIDTH-1:0] **data_in** : input data to be written to FIFO Memory
- **output** logic [DATA_WIDTH-1:0] **data_out** : data read out from FIFO Memory
- **output** logic **fifo_full** : indicates FIFO is full and there are no locations inside FIFO memory for further writes
- **output** logic **fifo_empty** : indicates FIFO is empty and there are no data available inside FIFO memory for reading
- **output** logic **fifo_almost_full** : one cycle early indication of FIFO_FULL (fifo is not full yet, it will be next cycle)
- **output** logic **fifo_almost_empty** : one cycle early indication of FIFO_EMPTY (fifo is not empty yet, it will be next cycle)

❑ Name of the asynchronous fifo module : async_fifo

❑ Use below mentioned modules provided :

- **shift_register** :
 - To implement 2-FF synchronizer for write and read pointer
 - To implement 1 bit delayed version of early fifo empty and fifo full
- **dual_port_ram** :
 - For FIFO Memory Implementation



Homework-9b : Asynchronous FIFO

❑ Homework Report should include :

- SystemVerilog design code snapshot for `async_fifo`, `shift_register` and `dual_port_ram` modules
- Synthesis resource usage snapshot generated from Quartus for `async_fifo` top level module
- Simulation snapshot and explain simulation results of `async_fifo`
 - Describe how data is sent to `async_fifo` and read from `async_fifo`. Explain how `async_fifo` works.
 - Explain write and read pointer synchronization how it is done
 - Explain how full and empty flags are generated
 - Explain role of `dual_port_ram` in `async_fifo` design and how read and write operation is performed to `dual_port_ram`
- Explanation of FPGA resource usage in report is not required.

❑ Lab9 folder includes :

- `async_fifo` folder which includes
 - Design template code for `async_fifo` module
 - Full design code for `dual_port_ram` and `shift_register`
 - Full testbench code for `async_fifo`
- For learning purpose, student can change the stimulus in initial block in testbench files

Homework-9b : Aysnchronous FIFO

- ❑ **Note** : First simulate Synchronous FIFO design with testbench provided and review waveform and RTL code to understand Synchronous FIFO design

- ❑ And then, start implementing asynchronous fifo RTL model

- ❑ Lab folder has :
 - Synchronous FIFO full RTL model with testbench and
 - For asynchronous FIFO, template RTL model code with steps are provided including partial code to guide students on implementation.

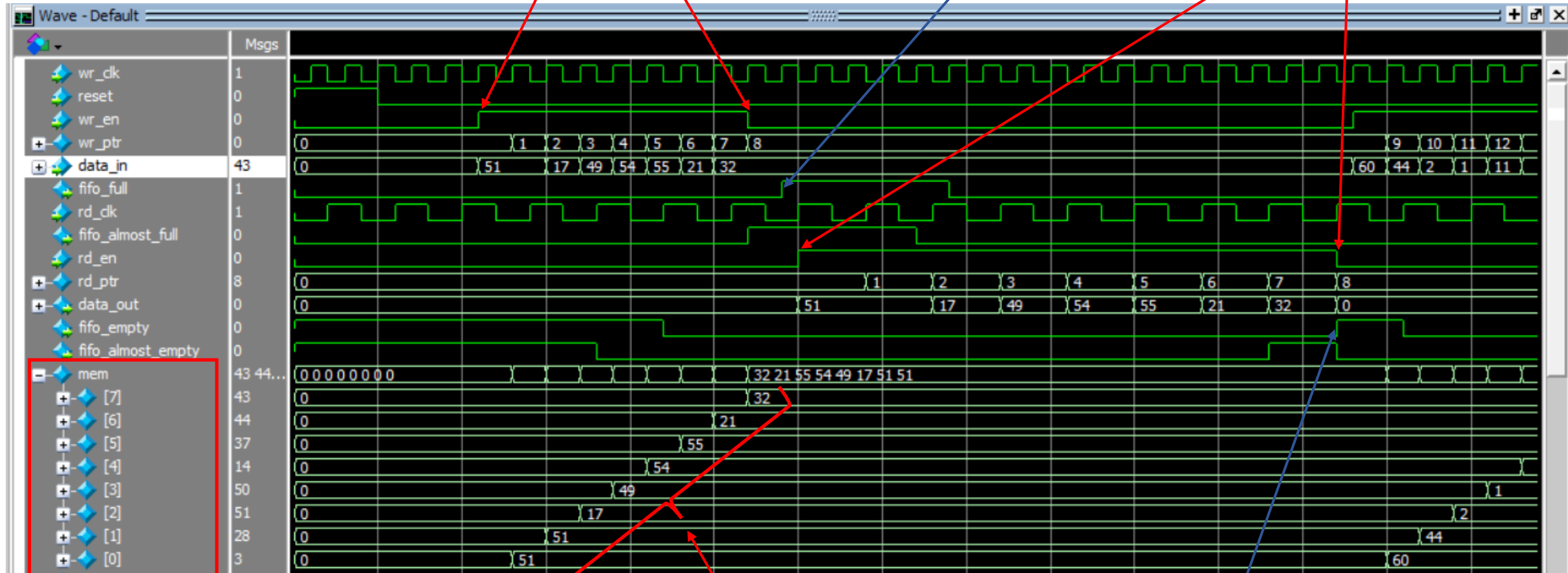
- ❑ **Note** :
 - **HW9 submission is optional and this will not have impact to final grade !**

Asynchronous FIFO Simulation Snapshot -1

when $wr_en == 1$ and $fifo_full == 0$
datain values : 51, 17, 49, 54, 55, 21, 32 are
stored in FIFO Memory

After writing 8 data
elements
 $fifo_full = 1$ is set

when $rd_en == 1$ and $fifo_empty == 0$
datain values : 51, 17, 49, 54, 55, 21, 32
are read out from FIFO Memory

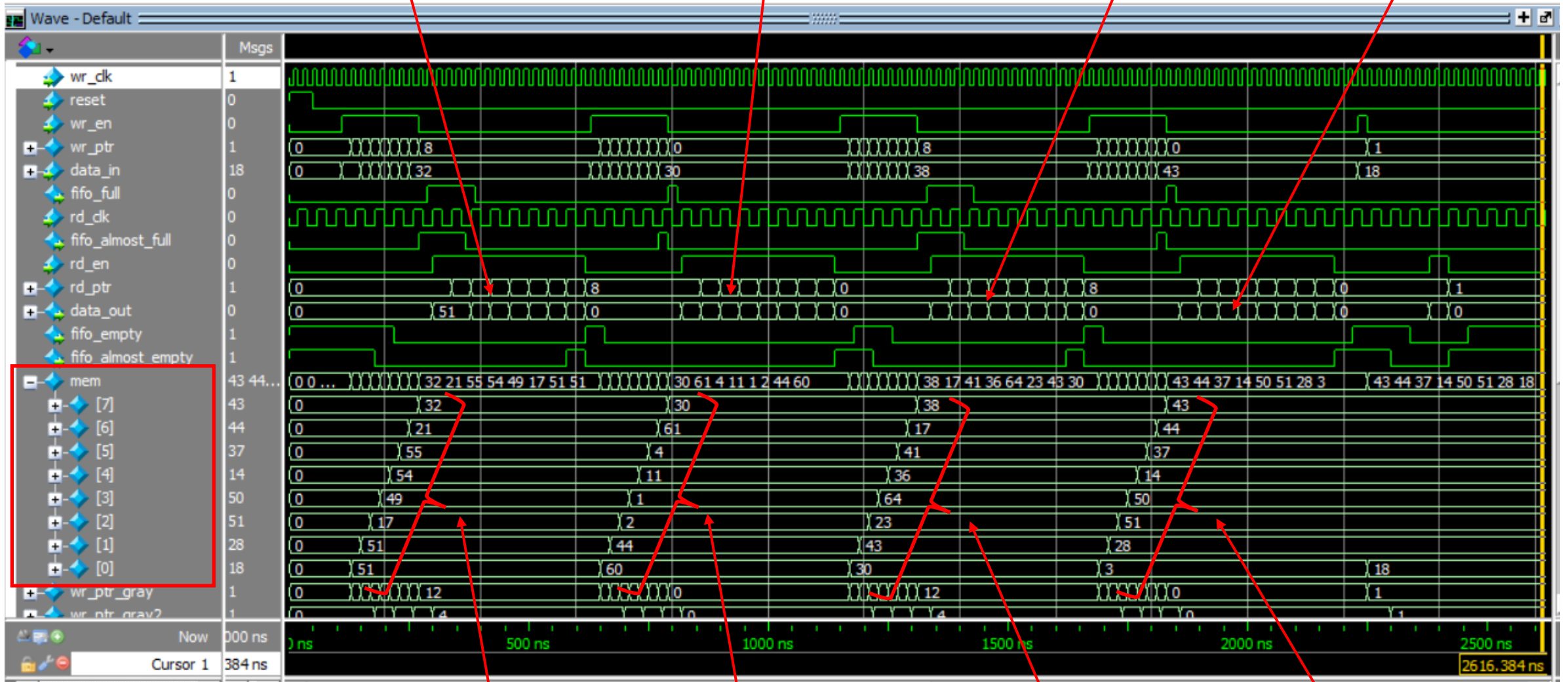


After reading all 8 data elements $fifo_empty = 1$ is set

FIFO Memory (mem) has datain values : 51, 17, 49, 54, 55, 21, 32. Each $rd_en=1$ caused data to push into FIFO memory 31

Asynchronous FIFO Simulation Snapshot -2

first set of 8 data values popped out of FIFO memory second set of 8 data values popped out of FIFO memory third set of 8 data values popped out of FIFO memory Fourth set of 8 data values popped out of FIFO memory



first set of 8 data values pushed in FIFO memory Second set of 8 data values pushed in FIFO memory Thrid set of 8 data values pushed in FIFO memory Fourth set of 8 data values pushed in FIFO memory

Steps to Model Asynchronous FIFO

- ❑ **Step-1** : Increment write pointer each time wr_en is '1' (binary counter)
- ❑ **Step-2** : Convert write pointer to gray value from binary write pointer value before sending write pointer to rd_clk domain through 2-FlipFlip synchronizer
- ❑ **Step-3** : Increment read pointer each time rd_en is '1' (binary counter)
- ❑ **Step-4** : Convert read pointer to gray value from binary read pointer value before sending read pointer to wr_clk domain through 2-FlipFlip synchronizer
- ❑ **Step-5** : Generate fifo empty flag using synchronized write pointer and original read pointer value comparison. Before comparison convert back synchronized write pointer gray value to binary value.
 - The FIFO is empty when both read and write pointers point to the same location
 - assign t_fifo_empty = (wr_ptr_binary2 == rd_ptr) ? 1 : 0;
- ❑ **Step-6** : Assert output signal fifo_almost_empty
 - FIFO Almost empty is generated simultaneously when the very last data available in fifo is read hence it is named as fifo almost empty. In another words, one cycle before fifo is actually empty
 - assign fifo_almost_empty = t_fifo_empty;

Steps to Model Asynchronous FIFO

- ❑ **Step-7** : Generate fifo full flag. . Before comparison convert back synchronized read pointer gray value to binary value.
 - FIFO is full when $wr_ptr - rd_ptr = 2^{address_width}$.
 - In that case, the Lower address bits are identical, but the MSB address bit is different.
 - assign `t_fifo_full = ((wr_ptr[ADDR_WIDTH] != rd_ptr_binary2[ADDR_WIDTH])`
▪ `&& (wr_ptr[ADDR_WIDTH-1:0] == rd_ptr_binary2[ADDR_WIDTH-1:0])) ? 1 :`
- ❑ **Step-8** : Assert almost full flag
 - FIFO Almost full is generated simultaneously when the very last location in fifo is written with data hence it is named as fifo almost full. In another words, one cycle before the fifo is actually full
 - assign `fifo_almost_full = t_fifo_full;`
- ❑ **Step-9** : Instantiate or create dual port single clock FIFO Memory
 - Memory should support synchronous write and asynchronous read
 - when connecting below `wr_en` and `rd_end` remember to do logical-and with `!fifo_full` and `!fifo_empty` respectively. This is to prevent FIFO overflow and underflow conditions
 - Dual port ram should support perform simultaneous write and read operations.

Steps to Model Asynchronous FIFO

- ❑ **Step-10** : Synchronize write pointer gray value to read clock domain using 2-FF synchronizer.
 - This is done to synchronize wr_ptr to rd_clk domain, and in rd_clk domain, output of this synchronizer will be used to compute fifo empty flag.
- ❑ **Step-11** : Convert synchronized write pointer gray value available from Step-10, back to binary value
 - Prior to generation for fifo empty flag, synchronized gray write pointer value is converted first to binary write pointer value
 - assign wr_ptr_binary2 = gray_to_binary(wr_ptr_gray2);
- ❑ **Step-12** : Synchronize read pointer gray value to write clock domain using 2-FF synchronizer
 - This is done to synchronize rd_ptr to wr_clk domain, and in wr_clk domain, output of this synchronizer will be used to compute fifo full flag.
- ❑ **Step-13** : Convert synchronized read pointer gray value available from Step-12, back to binary value
 - Prior to generation for fifo full flag, synchronized gray read pointer value is converted first to binary read pointer value
 - assign rd_ptr_binary2 = gray_to_binary(rd_ptr_gray2);

Steps to Model Asynchronous FIFO

- ❑ **Step-14** : Delay fifo almost empty ($t_{\text{fifo_empty}}$) by 1 clock cycle to generate fifo_empty output signal
 - This is done to generate fifo_empty output signal after the last available data in FIFO is read out
- ❑ **Step-15** : Delay fifo almost full ($t_{\text{fifo_full}}$) by 1 clock cycle to generate fifo_full output signal
 - This is done to generate fifo_full output signal after the last available location in FIFO is written with a data